Session 6C: User Management and Notification Portal

Trainer Reference Guide

Purpose and outcomes

Integrate Python fundamentals, validation, collections, functions, OOP, HTTP, templates, SQLite, CRUD, email, environment variables, and tests in an administrative application.

Architecture walkthrough

Browser/API client -> FastAPI router -> UserService -> UserRepository -> SQLAlchemy/SQLite. UserService also composes EmailService, which uses SMTP when completely configured and writes a timestamped text message otherwise. Schemas validate boundary input; the SQLAlchemy model represents persisted state.

Required project workflows

- Dashboard: total, active, inactive, department count, and five recent users.
- HTML: register, confirmation, searchable table, details, edit, delete with confirmation.
- JSON API: list/detail/create/update/delete with 200/201/204/404/409/422 behavior.
- User fields: names, email, department, role, active, creation timestamp, plus optional phone, employee ID, comments.
- Operations: initialize data/app.db, keep credentials in .env, log simulated email under logs/emails, run pytest.

Exercise/checkpoint sequence

1. Launch ‘/health‘ and ‘/docs‘; identify every layer for GET ‘/api/users‘.
2. Create a user in Swagger, then find it in the HTML list and database-backed dashboard.
3. Search by each supported field; edit status/department; inspect detail output.
4. Delete the record and verify both UI and API return the expected result.
5. Run without SMTP, inspect the email text file, and explain the fallback.
6. Run tests/test_users.py and tests/test_email_service.py; trace one failure if introduced.

Trainer key points specific to this module

Run Uvicorn from ‘final_project‘, never expose ‘.env‘, and keep SQL outside handlers. Trace one request end to end before learners change code. Distinguish model (database), schema (boundary contract), repository (persistence), service (workflow), and router (HTTP). Assess functionality first, then readability and presentation.

Completion evidence

All HTML/API CRUD paths work, invalid/duplicate input is handled, SQLite initializes, fallback email is readable, tests pass, and the learner can explain composition in UserService.

Delivery pattern

1. Explain the idea briefly and ask learners to predict a result.

2. Run the simple example, then connect it to an employee/user workflow.
3. Pair learners for the guided task; require an individual attempt next.
4. Use the challenge as the cumulative project increment.
5. Review the quiz and ask learners to explain, not only show, their code.

Trainer key points

- Keep live coding visible and deliberate. Normalize reading tracebacks.
- Ask what type and value each expression produces before running it.
- Let TODO files remain student-owned; open solutions only during review.
- Check boundary and failure cases, not only the happy path.
- End by tracing how the topic appears in the final portal.